

Hackathon 2026

Six Weeks of Building. One Night to Prove It.

You've spent six weeks learning to prompt, retrieve, reason, and deploy. This hackathon isn't a seventh lesson — it's the arena. Pick a problem statement and build something that genuinely works. No new frameworks to learn. No surprise tech. Just everything you already know, assembled under pressure.

36h

BUILD WINDOW

1-2

PER TEAM

5

PROBLEM STATEMENTS

5

WEEKS IN SCOPE

Hackathon Overview

This is the culmination of GenAI Summer of Code.

FORMAT

- Teams of **1-2** members.
- **36 hours** from kickoff to final submission.
- Choose **one** problem statement (PS1–PS5) at kickoff. Switching later costs you build time, so choose deliberately.

SUBMISSION REQUIREMENTS

- **Public GitHub repository** with a clear README (setup steps, architecture description, known limitations).
- **A live demo URL** — a deployed Hugging Face Space or a Gradio share link active at judging time.
- **A 5-minute demo video** walking through the problem, architecture, and a live run of your test scenarios.

NOT ALLOWED

-  Pre-built agent SaaS products or no-code builders.

Rules & Format

- **Code must be written during the 36-hour window.** Prior personal boilerplate is fine; entire pre-built apps are not.
- You may use AI coding assistants for implementation — the architecture and problem-solving must be your team's own.
- Open-source libraries and free-tier APIs only.
- If your live demo URL goes down before judging, your GitHub repo + video are the fallback.
- Plagiarized or templated submissions will be disqualified.

Evaluation Rubric

Every submission is scored out of 100 across five weighted criteria. Judges will run your test scenarios live where applicable.

Concept Utilisation		30%
<i>How deliberately and correctly you used the weekly techniques — not just "does it call the API" but "is RAG actually grounding the answer, is the agent actually routing."</i>		
Working Demo		25%
<i>The app runs, on the live URL, without crashing, for the judges' own inputs — not just your rehearsed demo path.</i>		
User Experience		20%
<i>Clear layout, sensible loading states, readable output, and graceful handling of bad input. Judges should never see a raw traceback.</i>		
Creativity & Usefulness		15%

Would a real person want this? A clever stretch goal, a thoughtful workflow, or a genuinely useful default all count here.

Video Presentation



10%

Clarity of explanation and demo pacing in the 5-minute video — not production value.

Tie-breaking: Ties are broken first by Concept Utilisation score, then by Working Demo score. If still tied, the team that handled more test scenarios correctly during live judging wins.

PS1 — EchoScribe

AI Meeting Notes Distiller

DIFFICULTY: EASY

Every meeting ends the same way: a wall of notes that nobody revisits, action items that live only in someone's memory, and a follow-up email that never gets written. Build the tool that transforms raw meeting notes into structured, actionable intelligence — in seconds.

WHAT TO BUILD

- a** Accept raw meeting notes or a transcript as text input (paste or file upload).
- b** Use prompt chaining to run three sequential passes: (1) extract a structured summary, (2) identify action items with owners and deadlines, (3) generate a ready-to-send follow-up email draft.
- c** Output each pass as structured JSON with clearly defined fields: `summary`, `decisions`, `action_items` (each with `task`, `owner`, `deadline`), `risks`.
- d** Offer three output tones — Formal, Casual, Executive — each driven by a genuinely distinct system prompt, changing vocabulary and structure, not just formality level.
- e** Display results in a clean Gradio Blocks UI with separate output panels for the summary, action items table, and email draft.

TECHNICAL REQUIREMENTS

- Prompt chaining: three separate, sequential Groq API calls — not one mega-prompt trying to do everything at once
- Structured JSON output for action items (parsed and rendered as a readable table in the UI)
- Three distinct system prompts for the three tone modes
- Chain-of-thought reasoning visible in at least one of the three passes (e.g. the risk-detection pass should show its reasoning before the final output)
- Gradio Blocks with `gr.Progress()` to show which pass is running

TEST SCENARIOS

- 01** Paste a messy 300-word meeting transcript. The action items output must include at least the tasks explicitly mentioned in the notes, each with a plausible owner if named in the transcript.
- 02** Run the same notes in Formal vs. Casual tone. The email draft must differ meaningfully in voice and structure — not just a synonym swap.
- 03** Paste notes from a meeting where no deadlines were mentioned — the action items JSON should have `deadline: null` or `"TBD"`, not a hallucinated date.
- 04** Submit an empty text box — the app must show a clear validation message, not an API error or silent failure.

WHAT JUDGES WILL LOOK FOR

- ✓ Three prompt passes run sequentially, not in one call
- ✓ Tone modes produce genuinely different outputs
- ✓ Action item JSON is valid and parses cleanly
- ✓ Progress feedback visible while the chain runs

TECH STACK

Groq API

LangChain

Gradio Blocks

✦ **STRETCH GOAL** Add a few-shot prompting panel where users can provide one example of a "good" action item format, and the app adapts all future extractions to match that style — demonstrating runtime few-shot conditioning.

PS2 — StudyBuddy Pro

AI Study Assistant with RAG

DIFFICULTY: MEDIUM

Every student has the same exam-week problem: a folder of PDFs and no way to test whether they actually understand them. Build the tool that turns passive notes into active recall — and then tells the student exactly what they still don't know.

WHAT TO BUILD

- a** Accept multiple uploaded study documents (PDF or text), indexed into a single searchable vector store.
- b** Answer free-text questions about the notes with section-level citations (document name + page number) — every answer must be traceable to a chunk.
- c** Generate a 10-question quiz (5 MCQ + 5 short-answer) sourced entirely from the uploaded content via structured JSON output — no general-knowledge filler.
- d** After the student submits answers, identify which topics were missed, retrieve the relevant chunks again, and generate a focused 2-paragraph revision summary per weak topic.

TECHNICAL REQUIREMENTS

- ChromaDB with `RecursiveCharacterTextSplitter` (chunk_size=500, overlap=100)
- Embeddings via `all-MiniLM-L6-v2` (HuggingFaceEmbeddings)
- Quiz generation must use structured JSON output, not free-form text parsing
- Revision summaries must quote/paraphrase only from retrieved chunks — no hallucinated facts

TEST SCENARIOS

- 01** Upload a 10-page lecture PDF and ask a question whose answer appears only on page 7 — the citation must correctly point to page 7.
- 02** Generate the quiz twice from the same document — all 10 questions both times must be answerable from the uploaded text, not outside knowledge.
- 03** Deliberately answer 3 of 10 quiz questions wrong — the app must flag those 3 topics and generate summaries specific to them, not generic ones.
- 04** Upload two unrelated documents and ask a question only the second answers — the system should not hallucinate an answer from the first.

WHAT JUDGES WILL LOOK FOR

- ✓ Quiz questions are verifiably grounded in the uploaded document
- ✓ Citations are accurate, not approximate
- ✓ Revision summaries address the specific missed topic
- ✓ App handles a document with no relevant answer gracefully

TECH STACK

LangChain ChromaDB sentence-transformers Groq API Gradio Blocks

✦ **STRETCH GOAL** Track quiz performance across multiple sessions (in-memory JSON log) and show a "topics you keep missing" trend — turning one quiz into a spaced-repetition signal.

PS3 — CodeMentor

Adaptive Code Explainer

DIFFICULTY: MEDIUM

Reading someone else's code is harder than writing your own. Build a mentor that meets a developer at their level, explains what's actually happening, and — unlike a static explainer — goes out and verifies its claims against real documentation.

WHAT TO BUILD

- a** Accept any code snippet (Python, JavaScript, Java, or C++) plus an explanation level: Beginner, Intermediate, or Expert.
- b** Explain the code at that level with a line-by-line breakdown and one analogy-based summary.
- c** Use an agent with a documentation-search tool to look up any library or API call detected in the snippet, and fold real findings into the explanation.
- d** Identify potential bugs and suggest specific, concrete fixes — not generic "consider adding error handling."
- e** Maintain conversation history so the user can ask natural follow-up questions about the same snippet.

TECHNICAL REQUIREMENTS

- Three genuinely distinct system prompts for the three levels — not one prompt with a tone adjective swapped in
- Structured JSON output (breakdown, analogy, bugs, doc-findings as separate fields)
- [DuckDuckGoSearchRun](#) as the agent's documentation-lookup tool
- Gradio UI with Accordion components keeping the breakdown, analogy, and bug list visually separated

TEST SCENARIOS

- 01** Paste the same 15-line Python snippet at all three levels — Beginner and Expert explanations must differ meaningfully in depth and vocabulary, not just length.
- 02** Paste a snippet importing a less-common library (e.g. [asyncio.gather](#)) — the agent must search and surface a real, relevant doc finding.
- 03** Paste a snippet with an intentional off-by-one bug — the bug list must catch it and propose a specific fix.
- 04** Ask a follow-up like "what would happen if I removed line 4?" — the agent must use the original snippet from memory, not ask the user to repaste it.

WHAT JUDGES WILL LOOK FOR

- ✓ Explanation quality varies meaningfully across levels
- ✓ Documentation search correctly identifies libraries
- ✓ Bug suggestions are specific and actionable
- ✓ Follow-up questions correctly use conversation memory

TECH STACK

LangChain Agents DuckDuckGo Search Groq API Gradio Blocks

✦ **STRETCH GOAL** Add a "Compare two languages" mode — paste the same logic in Python and JavaScript and get a side-by-side breakdown of idiomatic differences.

PS4 — ResearchMate

Hybrid Research Agent

DIFFICULTY: **HARD**

Real research never lives in one place. Some of it is in your own PDFs; the rest is changing on the web right now. Build the agent that knows which source to check for which kind of question — and is honest when the two disagree.

WHAT TO BUILD

- a** Accept multiple uploaded PDFs, indexed into a vector store.
- b** Accept a research question from the user.
- c** Use a hybrid agent with two tools — `search_documents` (RAG) and `search_web` (DuckDuckGo) — that decides per sub-question which source to consult.
- d** Produce a structured investigation report with citations clearly distinguishing document-sourced claims from web-sourced claims.
- e** Explicitly flag conflicts where the document and the web disagree.

TECHNICAL REQUIREMENTS

- Two custom tools built with the `@tool` decorator: `search_documents` over ChromaDB and `search_web` via DuckDuckGo
- Agent built on `create_react_agent`, routing to the correct tool per sub-question rather than always calling both
- Final output as structured JSON: `findings`, `conflicts`, `conclusion`
- Gradio UI with a clear three-part layout: document upload, question input, structured report display

TEST SCENARIOS

- 01** Ask a question entirely answerable from the PDF — the agent should answer without needlessly calling web search.
- 02** Ask a question with no answer in the documents but answerable on the web — the agent must fall back to web search correctly.
- 03** Upload a document with an intentionally outdated figure and ask the same question to both sources — the conflict section must surface the discrepancy.
- 04** Ask a multi-part question requiring both sources — the final report must cite both, attributed correctly to each.

WHAT JUDGES WILL LOOK FOR

- ✓ Agent correctly routes to RAG vs. web based on the query
- ✓ Conflict detection identifies genuine contradictions
- ✓ Citations clearly distinguish document vs. web sources
- ✓ Report stays structured even on messy multi-part questions

TECH STACK

LangGraph Agents ChromaDB DuckDuckGo Search Groq API Gradio Blocks

✦ **STRETCH GOAL** Show the agent's reasoning trace (via `agent.stream()`) in a collapsible panel so users can see exactly which tool fired for which sub-question.

PS5 — Multimodal Q&A Pro

Full Hybrid Agent + Deployment

DIFFICULTY: HARD

This is the capstone-of-the-capstone: every tool type from the program, in one agent, deployed live. It's the hardest problem on this list — choose it if your team wants to be judged on doing everything, not just doing one thing well.

WHAT TO BUILD

- a** Accept PDF uploads, indexed in ChromaDB.
- b** Accept image uploads, processed via Groq Vision.
- c** Accept free-text questions from the user.
- d** Run a single `create_react_agent` with three tools: `search_documents`, `search_web`, `describe_image`.
- e** Have the agent route intelligently to the right tool(s) per query.
- f** For complex queries, the agent uses multiple tools and synthesises one unified answer.
- g** Surface a reasoning trace (which tools were called, in what order) via `agent.stream()`, shown in a collapsible Gradio Accordion.
- h** Deploy as a multi-tab Gradio app: Tab 1 (Hybrid Chat), Tab 2 (Document Q&A), Tab 3 (Image Studio).
- i** Deploy live to Hugging Face Spaces with secrets stored properly — not hardcoded.

TECHNICAL REQUIREMENTS

- All three tools on one `create_react_agent` instance with `recursion_limit=12`
- Reasoning trace derived from real `agent.stream()` output — not a fabricated log
- Multi-tab Gradio layout with a clean design per tab
- Error handling on every tool path via a `@safe_call`-style decorator — a failed web search or vision call should degrade gracefully
- Live, working Hugging Face Spaces deployment with secrets management for API keys

TEST SCENARIOS

- 01** Ask a pure document question — only `search_documents` should fire, confirmed via the reasoning trace.
- 02** Upload an image and ask "what is this and is it mentioned in my documents?" — the trace must show `describe_image` followed by `search_documents`, in that order.
- 03** Ask a question requiring current information with no relevant documents uploaded — the agent should fall back cleanly to `search_web` alone.
- 04** Simulate a web search timeout — the app should report the failure in the UI, not crash the session.
- 05** Open the live HF Spaces URL cold and run a full query end-to-end — it must work without any local setup from the judge's side.

WHAT JUDGES WILL LOOK FOR

- ✓ Agent routes to the right tool(s) per query

- ✓ Multi-tool queries genuinely synthesise, not just concatenate
- ✓ Reasoning trace accurately reflects real tool calls
- ✓ App handles edge cases and tool failures gracefully
- ✓ Live URL works with proper API key management
- ✓ Multi-tab UI stays clean under real use

TECH STACK

LangGraph Agents

ChromaDB

Groq Vision API

DuckDuckGo Search

Gradio Blocks

HF Spaces

✦ **STRETCH GOAL** Add [WikipediaQueryRun](#) as a fourth tool for encyclopedic lookups, and have the agent justify in the trace why it chose Wikipedia over web search for a given sub-question.

Final Notes

Every one of these five problems was built using only what's already in your hands. There's no hidden technique, no last-minute library to learn at hour 3. The difference between a good submission and a great one won't be who knows more — it'll be who scoped tightly, tested honestly, and shipped something that actually runs when a judge clicks the link cold.

One last thing: pick the problem that excites you, not just the one that looks easiest on paper. Thirty-six hours goes faster when you actually want to see the thing work.

Note: We encourage you to use the weekly concepts and topics that you learned during the MSOC 6 week learning phase to use to the fullest. The problem statements are also designed keeping in mind those concepts. However, you are not limited to using only those topics/concepts. you are open to use different concepts which you feel will enhance your final project submission. Also, the tech stack mentioned is only for reference, you can use different tools/stack too.

Good luck. Build something real.